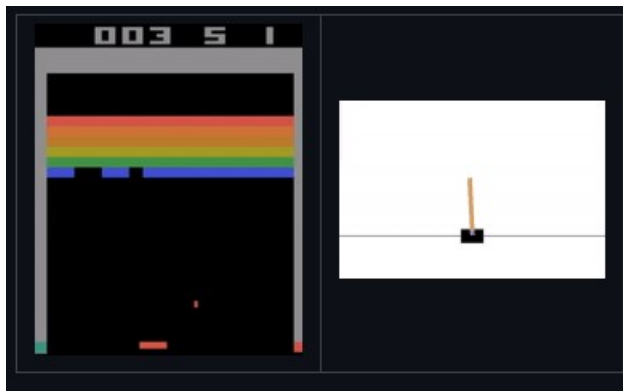
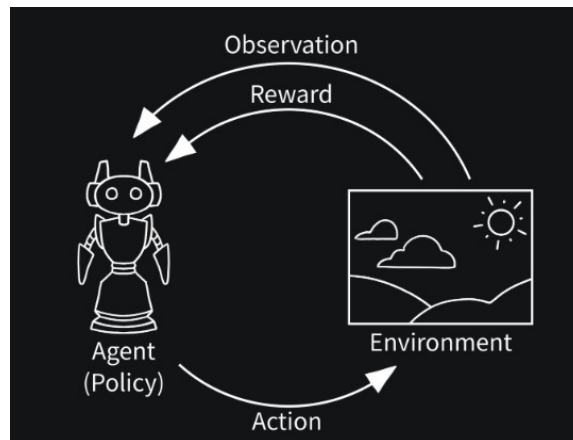


Reinforcement Learning with Tensorflow, Keras-RL and Gym



<https://github.com/keras-rl/keras-rl>



https://gymnasium.farama.org/introduction/basic_usage/

```
import time # to reduce the game speed when playing manually

import gym # Contains the game we want to play
from pygame.window import key # for manual playing

# Import necessary blocks from keras to build the Deep Learning backbone of our agent
from tensorflow.keras.models import Sequential # To compose multiple layers
from tensorflow.keras.layers import Dense # Fully-Connected Layer
from tensorflow.keras.layers import Activation # Activation functions
from tensorflow.keras.layers import Flatten # Flatten function

from tensorflow.keras.optimizers import Adam # Adam optimizer

# Now the keras-rl2 agent. Don't get confused as it is only called rl and not keras-rl
from rl.agents.dqn import DQNAgent # Use the basic Deep-Q-Network agent
```

kaggle

Credit Card Fraud Detection

Anonymized credit card transactions labeled as fraudulent or genuine

Data Card Code (5457)

About Dataset



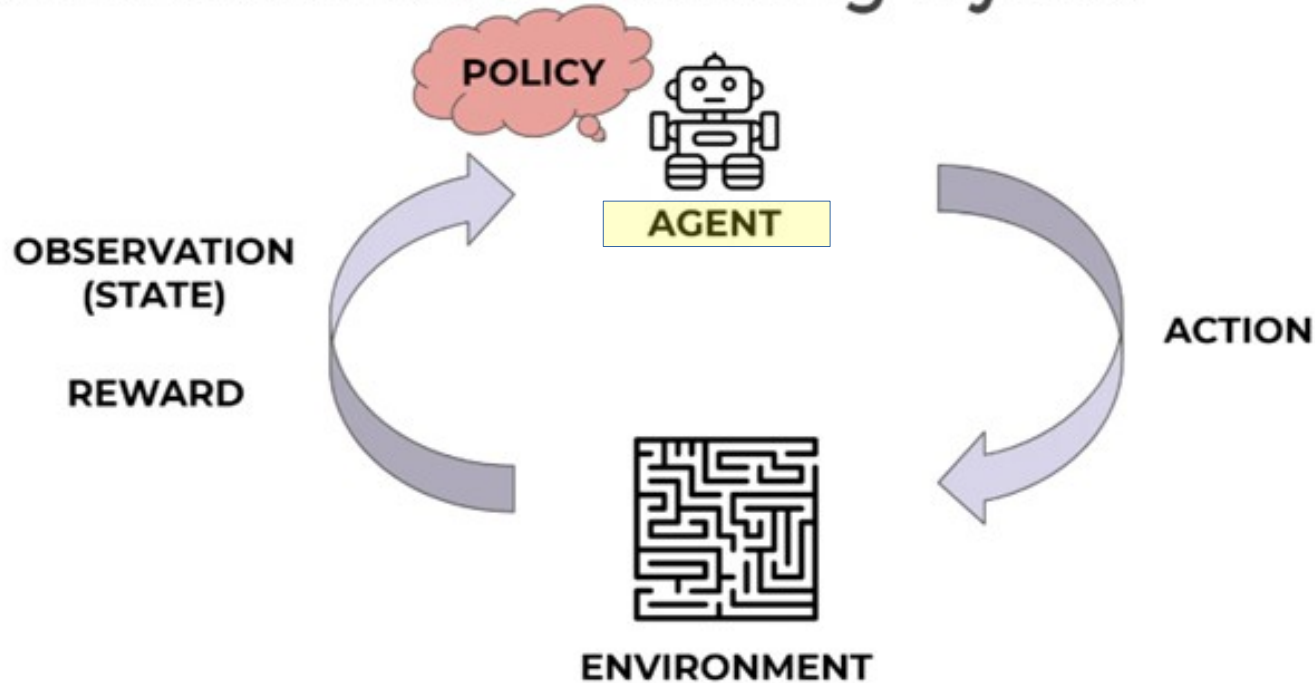
Usability

8.53

<https://www.kaggle.com/mlg-ulb/creditcardfraud>

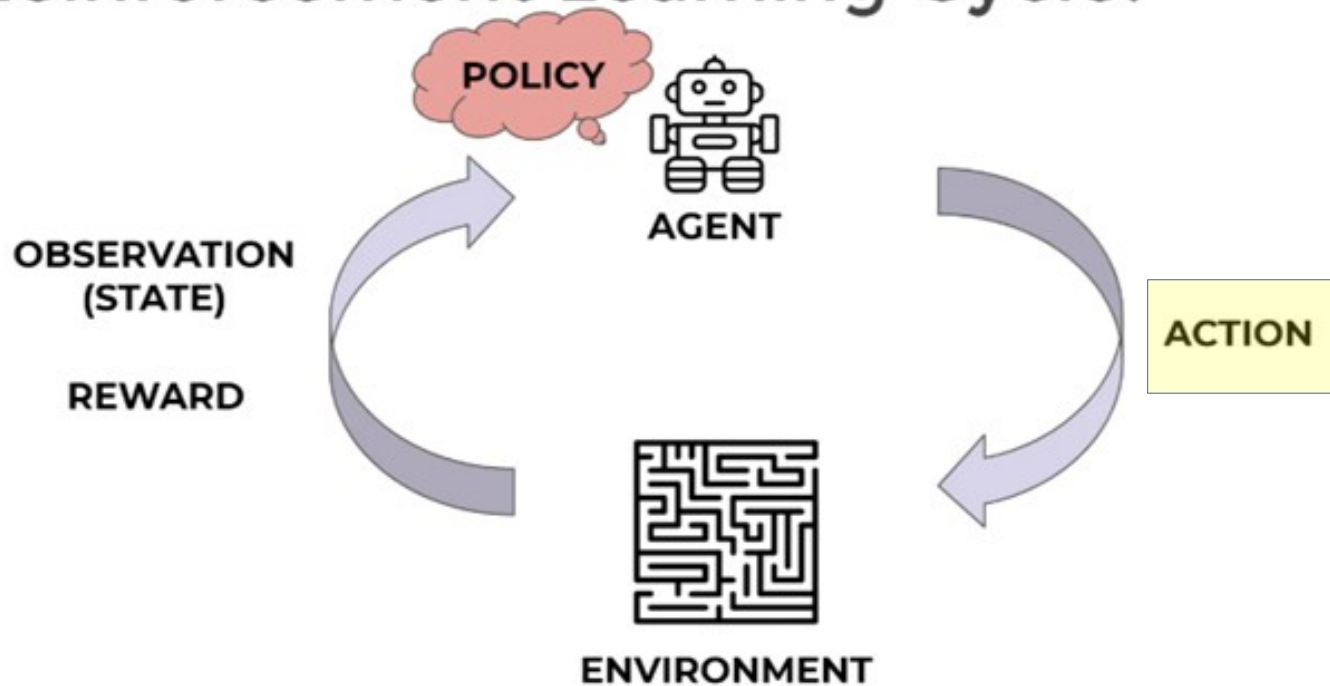
Reinforcement Learning

- Reinforcement Learning Cycle:



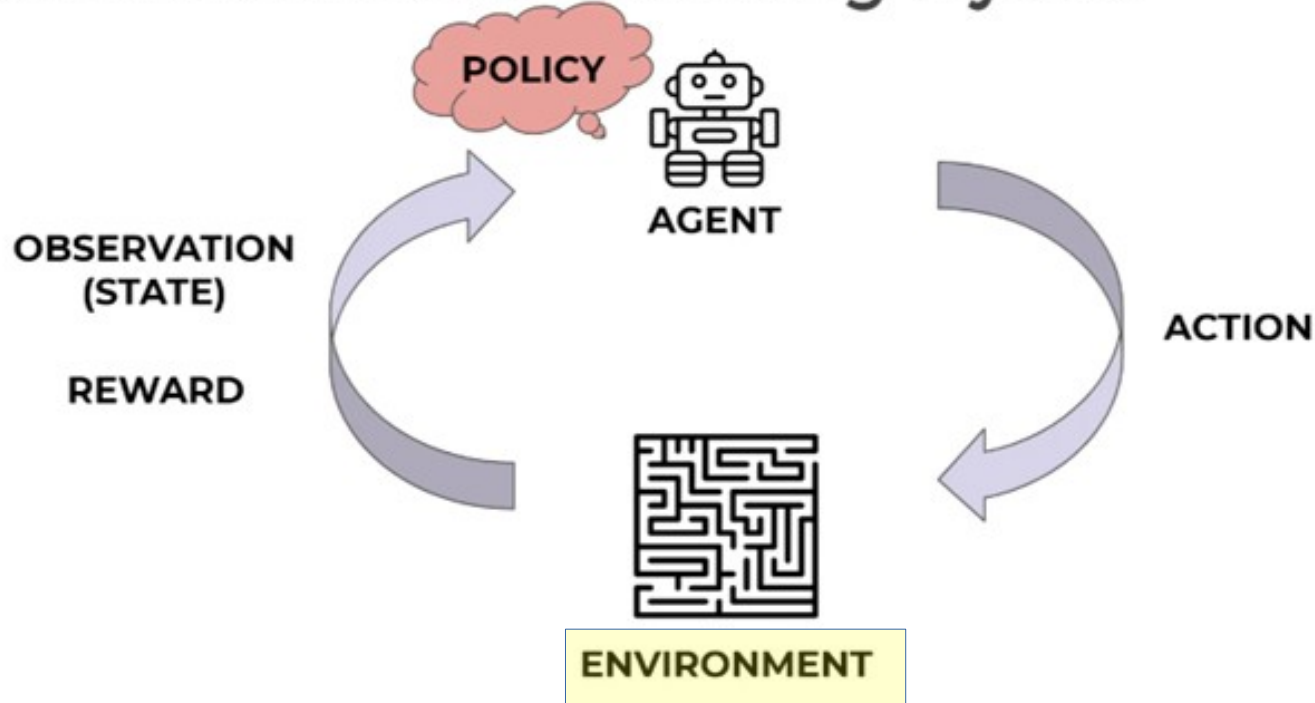
Reinforcement Learning

- Reinforcement Learning Cycle:



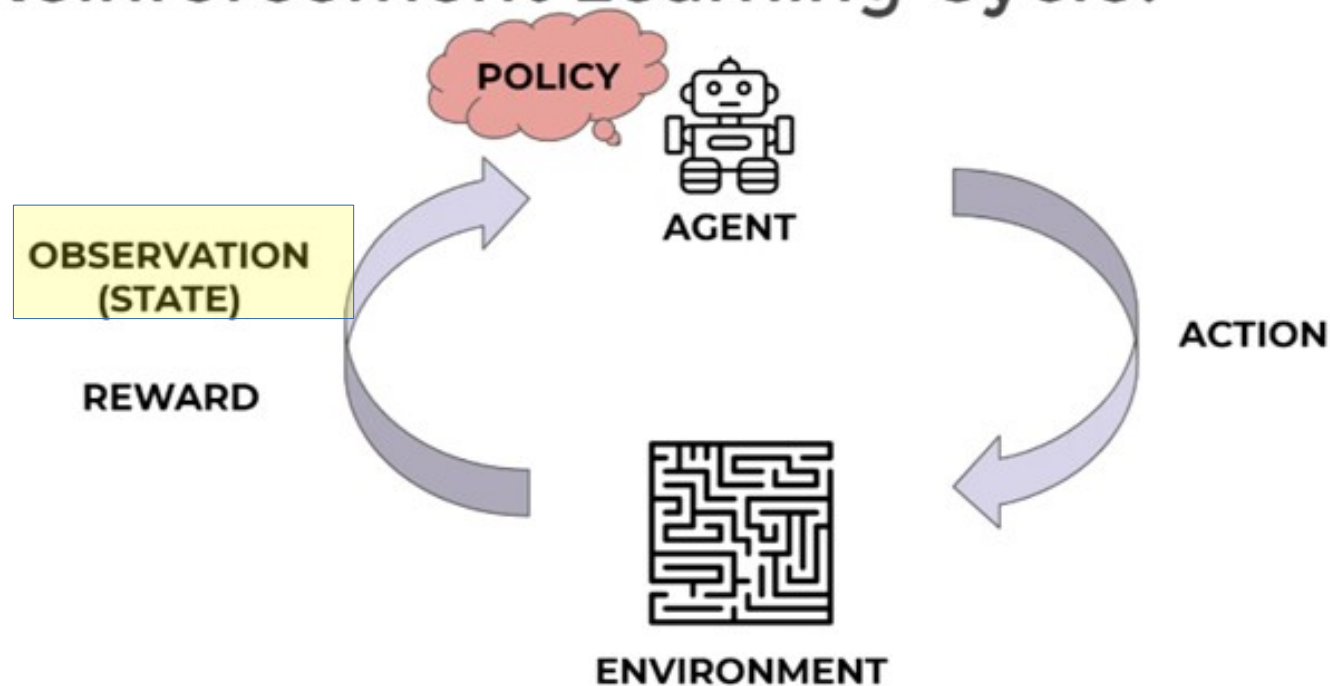
Reinforcement Learning

- Reinforcement Learning Cycle:



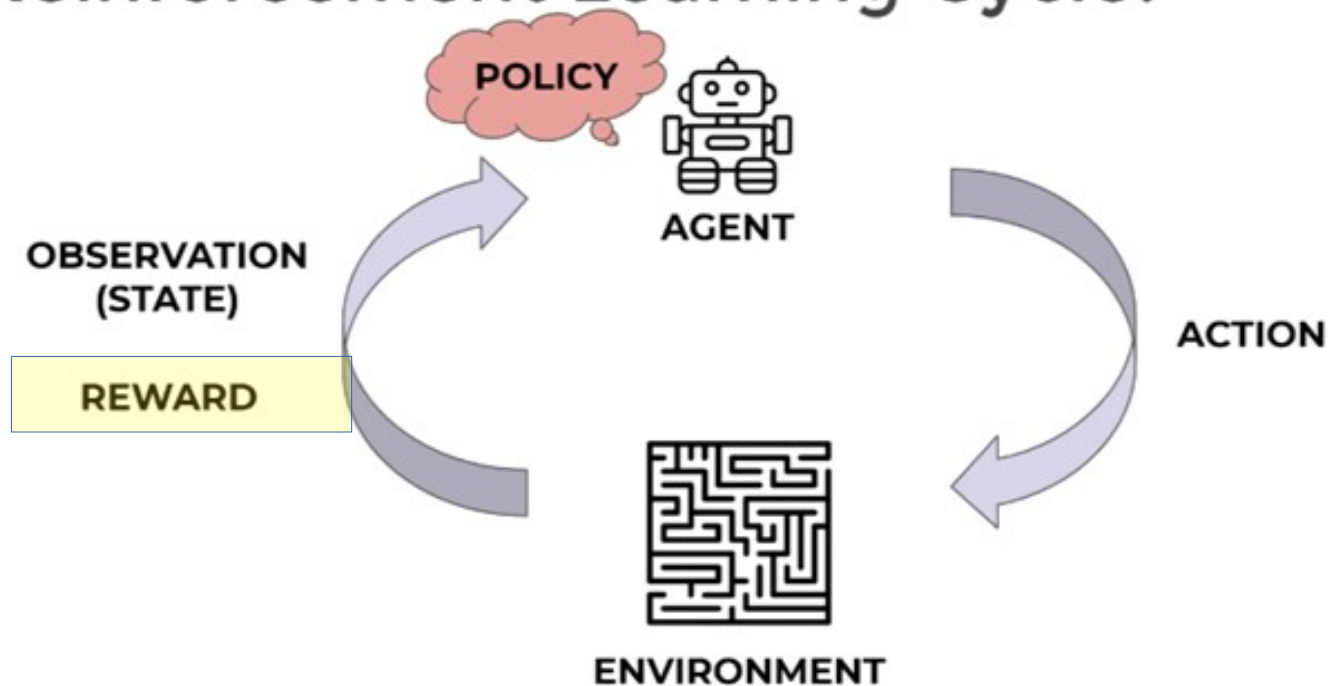
Reinforcement Learning

- Reinforcement Learning Cycle:



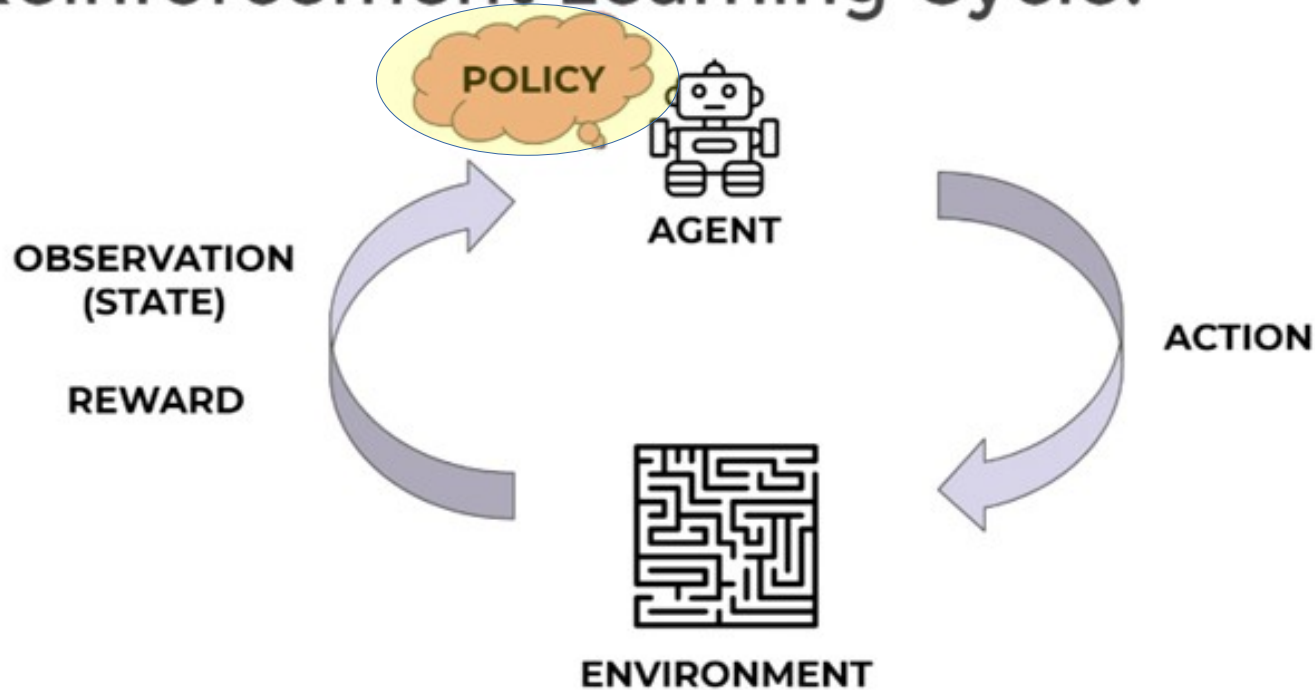
Reinforcement Learning

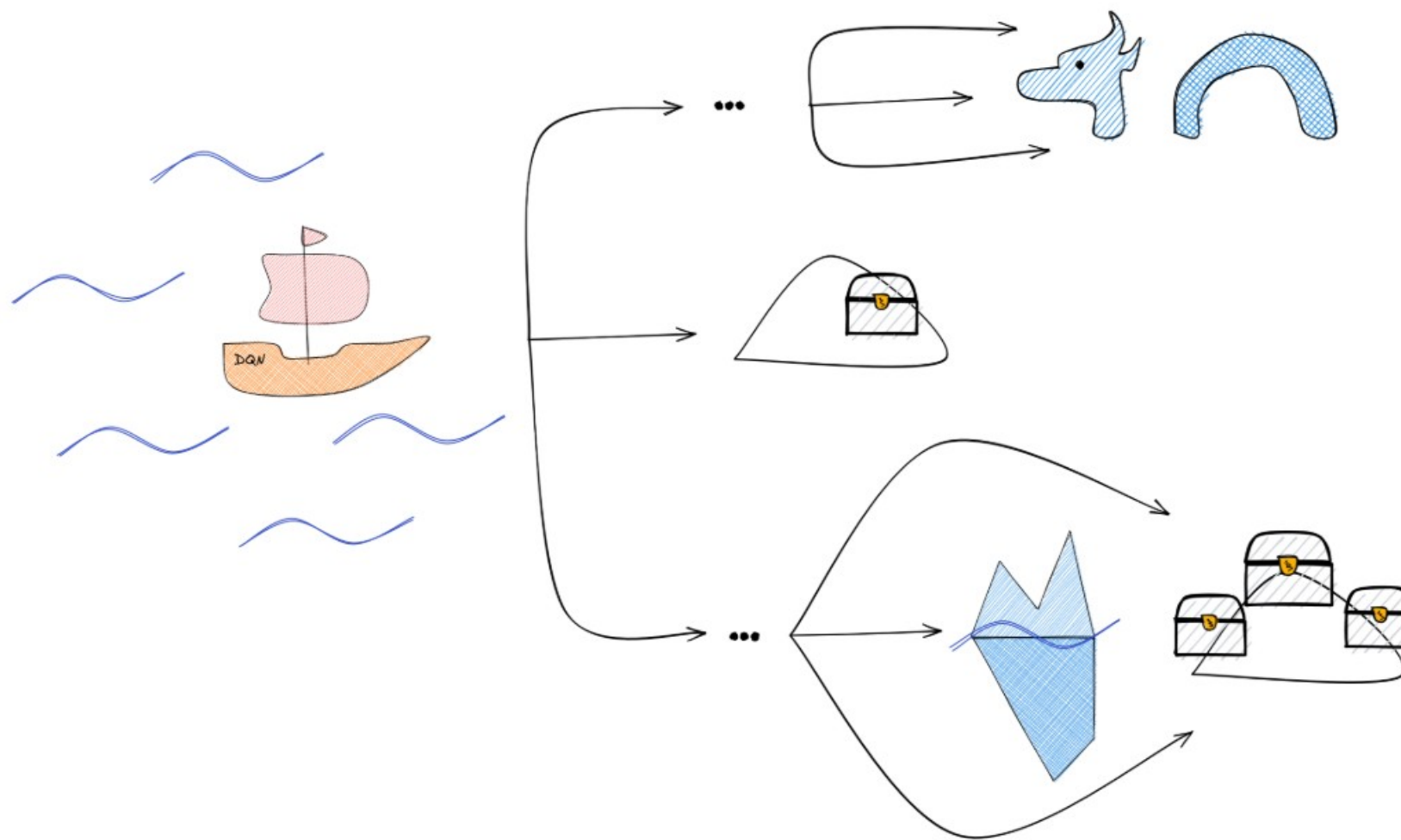
- Reinforcement Learning Cycle:

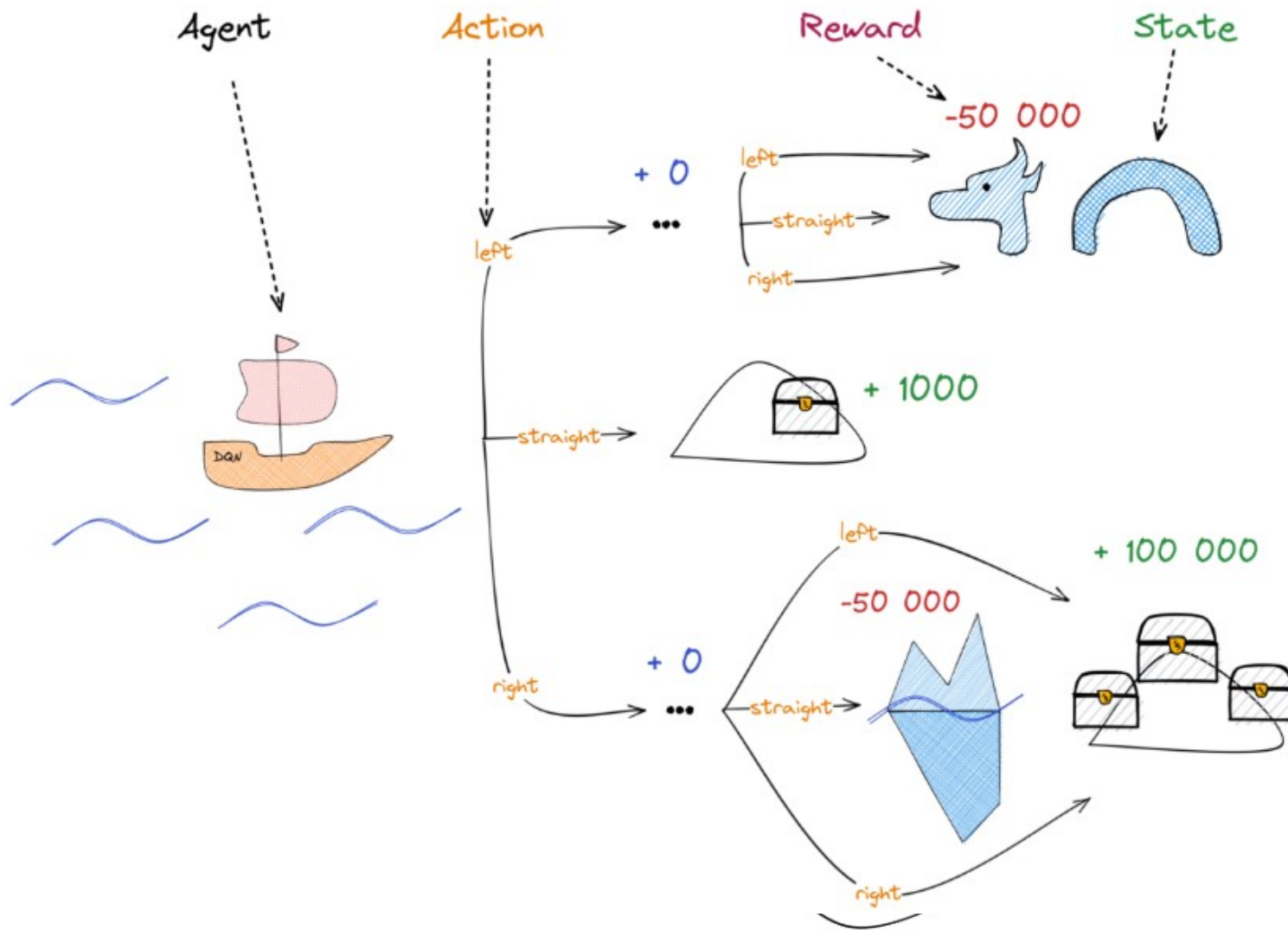


Reinforcement Learning

- Reinforcement Learning Cycle:







Agent

Action

Reward

State

+ 0

-50 000

left

straight

right

left

straight

+ 1000

left

-50 000

+ 100 000

+ 0

right

straight

right

ACTION

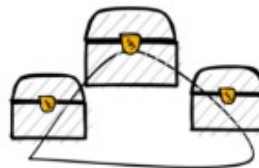
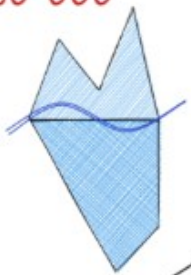
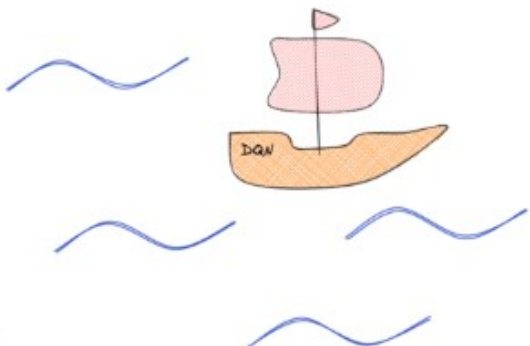
POLICY

AGENT

OBSERVATION
(STATE)

REWARD

ENVIRONMENT



Agent

Action

Reward

State

+ 0

-50 000

left

straight

right

left

straight

+ 1000

left

-50 000

+ 100 000

+ 0

right

straight

right

ACTION

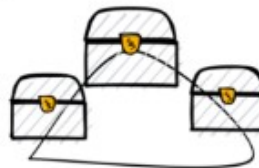
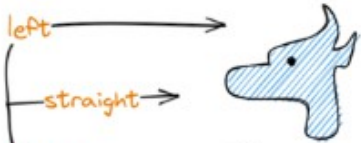
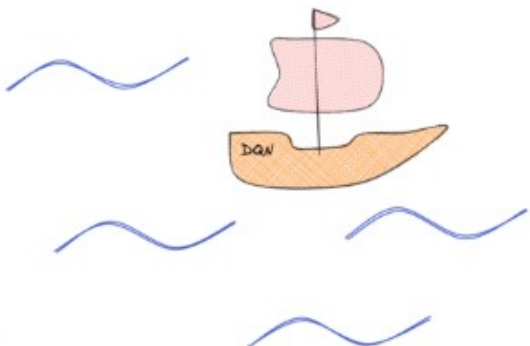
POLICY

AGENT

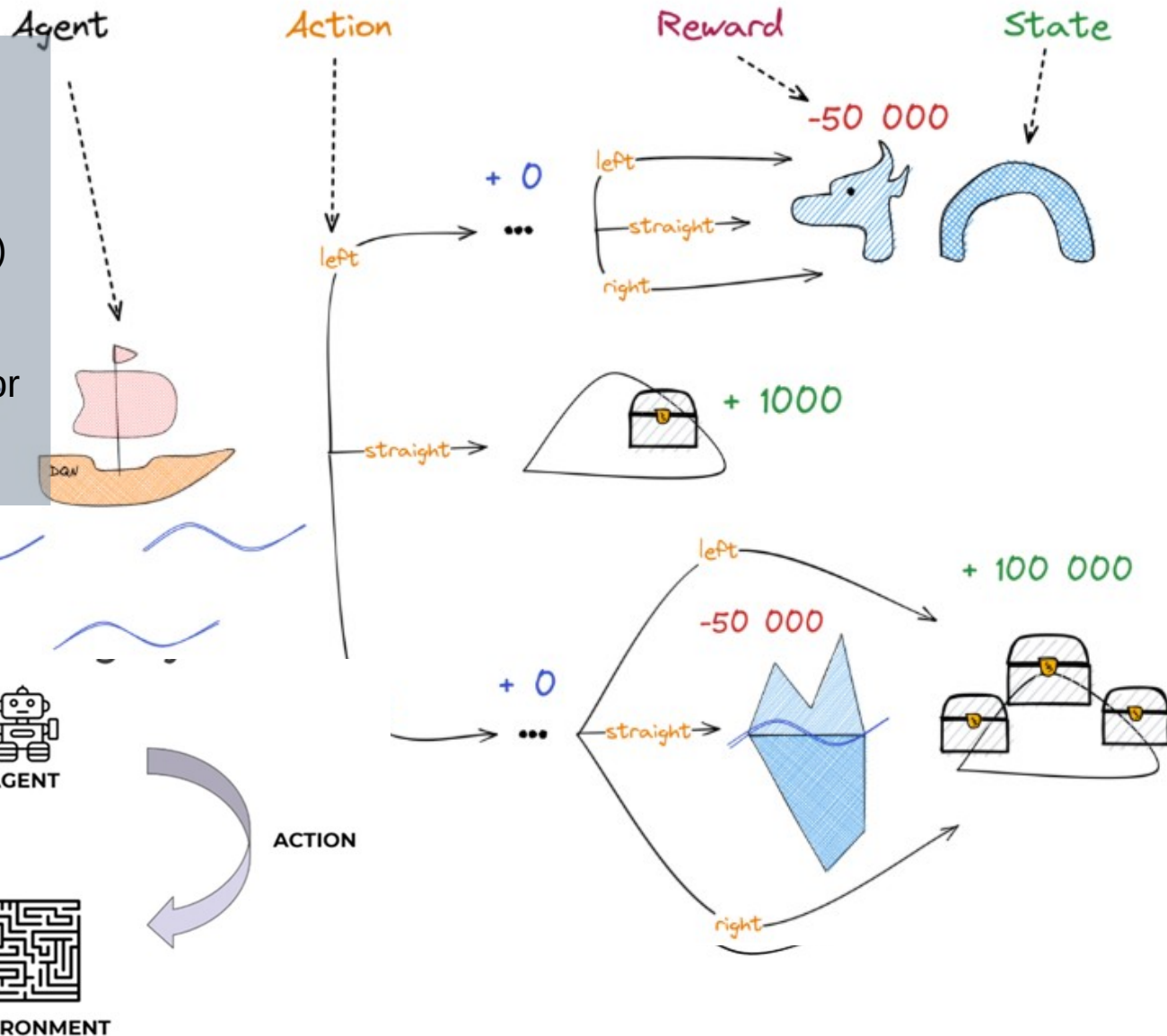
OBSERVATION
(STATE)

REWARD

ENVIRONMENT



- Agent
- Perform an action (policy driven by)
- It generates an observation ("State") from the environment.
- It gives a Reward (or penalty)
- Update policy

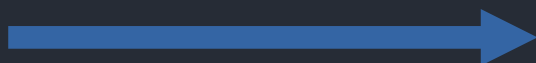


```
[11]: env = gym.make('FrozenLakeNotSlippery-v0')
env.reset() #8:49 Lec 96 Play 96. Q-Learning Implementation - Part One - Environment Setup
# https://www.udemy.com/course/practical-ai-with-python-and-reinforcement-learning/learn/lecture/27136050#questions

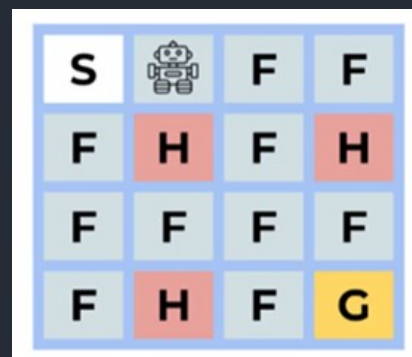
for step in range(5):
    env.render()
    action = env.action_space.sample()
    env.step(action)
    time.sleep(0.5)

env.close()
```

```
SFFF
FHFH
FFFH
HFFG
  (Up)
SFFF
FHFH
FFFH
HFFG
  (Right)
SFFF
FHFH
FFFH
HFFG
  (Right)
SFFF
FHFH
FFFH
HFFG
  (Left)
SFFF
FHFH
FFFH
HFFG
```

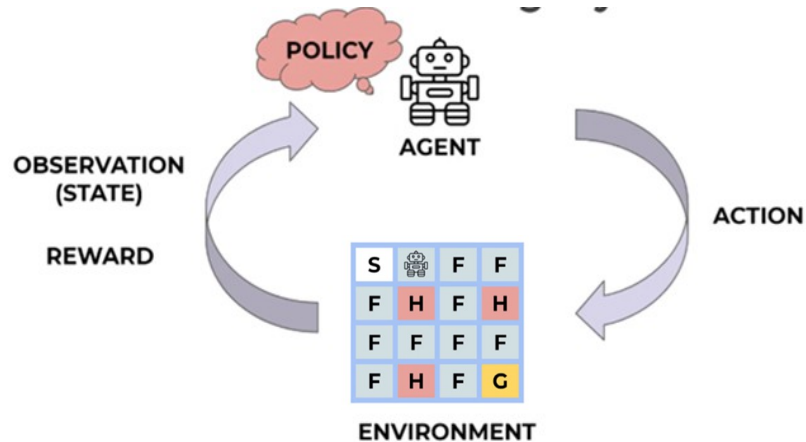


```
SFFF
FHFH
FFFH
HFFG
  (Up)
SFFF
FHFH
FFFH
HFFG
  (Right)
SFFF
FHFH
FFFH
HFFG
  (Right)
SFFF
FHFH
FFFH
HFFG
  (Right)
```



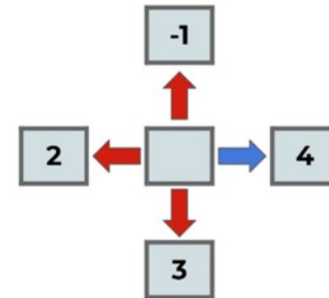
- Bellman Equation combines **rewards** and **discount factors** to define the value of a current state:

$$V(s) = \max_a (R(s, a) + \gamma V(s'))$$



- Deterministic Policy:

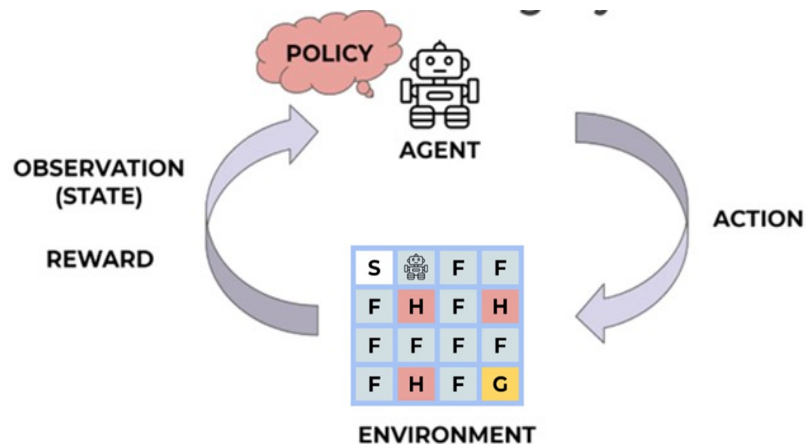
←	↓	→	↑
0	0	1	0



$$a_t = \mu(s_t)$$

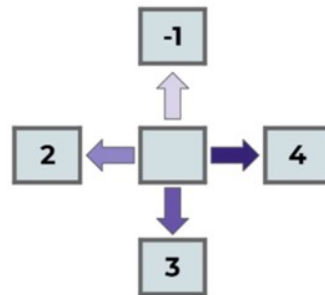
- Bellman Equation combines **rewards** and **discount factors** to define the value of a current state:

$$V(s) = \max_a (R(s, a) + \gamma V(s'))$$



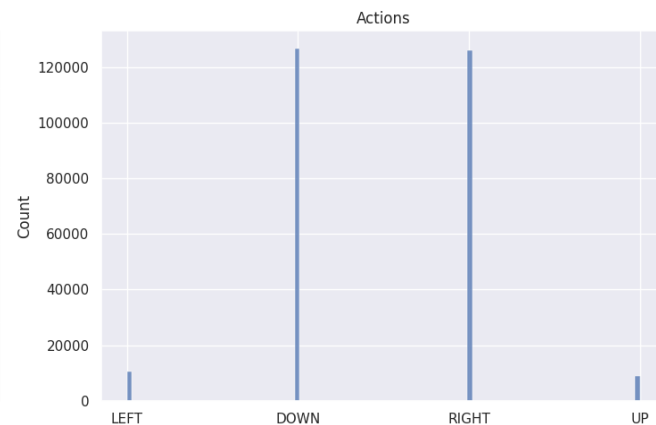
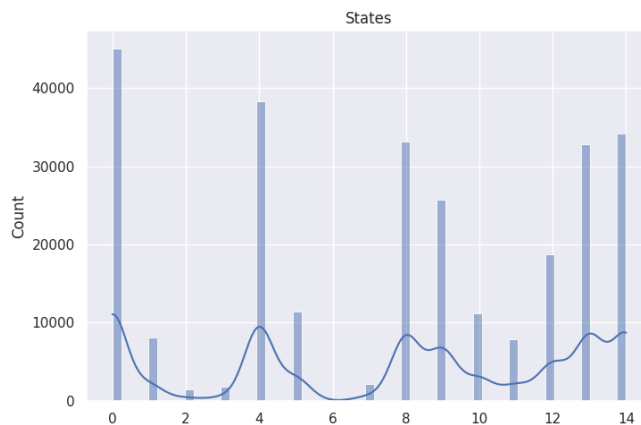
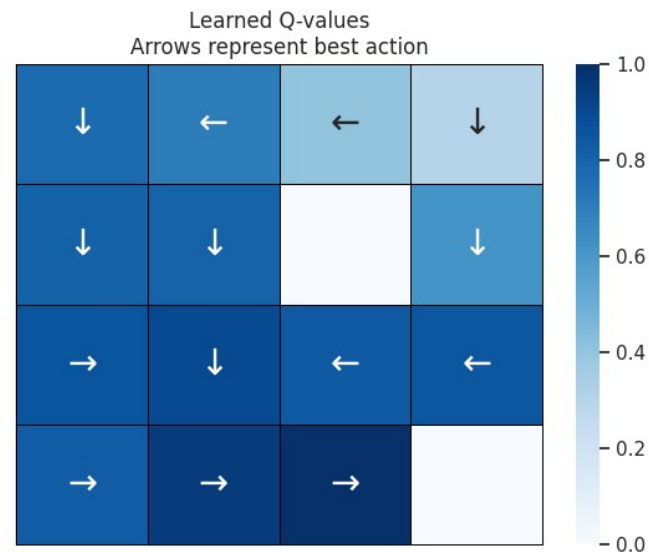
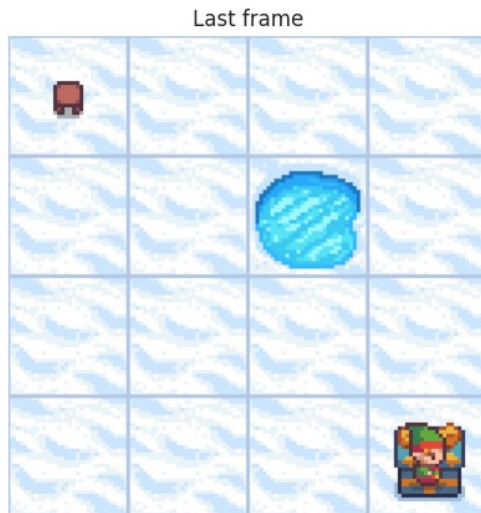
- Stochastic Policy:

←	↓	→	↑
0.2	0.3	0.4	0.1



$$\pi_s(A|S) = \pi_s(A|S = s_1), \dots, \pi_s(A|S = s_{|S|})$$

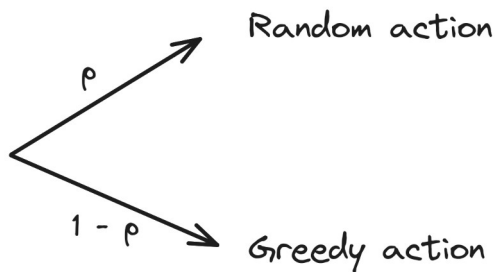
"Real"
toy
model.




```
def epsilon_greedy_action_selection(model, epsilon, observation):
    if np.random.random() > epsilon:
        prediction = model.predict(observation) # perform the prediction
        action = np.argmax(prediction) # Chose the action with the highest value
    else:
        action = np.random.randint(0, env.action_space.n) # Else use random action
    return action
```

Epsilon
greedy action
selection

**Exploration
Vs.
Exploitation**



p : probability to select a random action (aka epsilon)



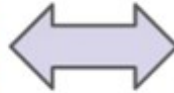
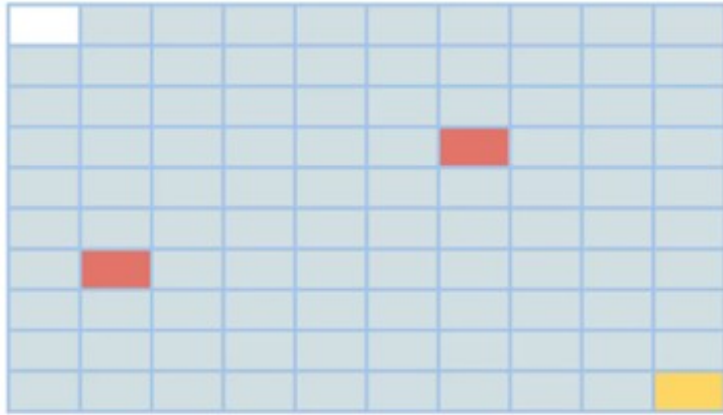
SFFF
FHFH
FFFH
HFFG
(Up)
SFFF
FHFH
FFFH
HFFG
(Right)
SFFF
FHFH
FFFH
HFFG
(Right)

S		F	F
F	H	F	H
F	F	F	F
F	H	F	G

Q-Learning

- What we've learned so far:
 - General Concepts:
 - Environments, Agents, Rewards, Observations, State, and Actions.
 - Manual Agents:
 - Simply use observations to complete a task.

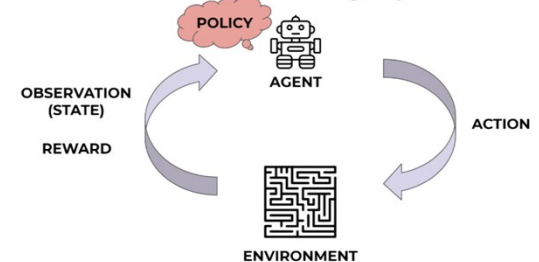
- Clearly not scalable for environments with high number of potential states!



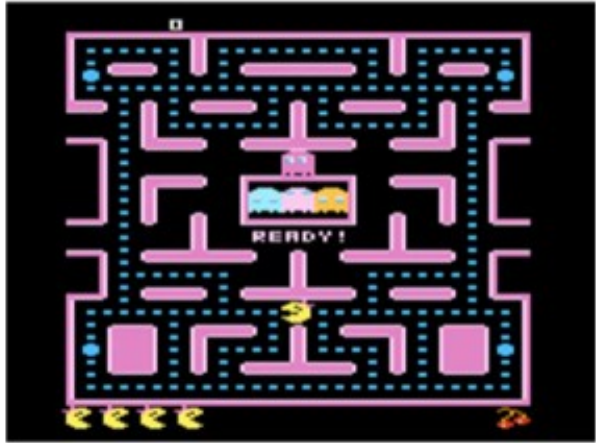
	Action 0	Action 1	...	Action N
State 0	Q	Q	...	Q
State 1	Q	Q	...	Q
...	...	...	...	...
State N	Q	Q	...	Q

Reinforcement Learning

- Reinforcement Learning Cycle:



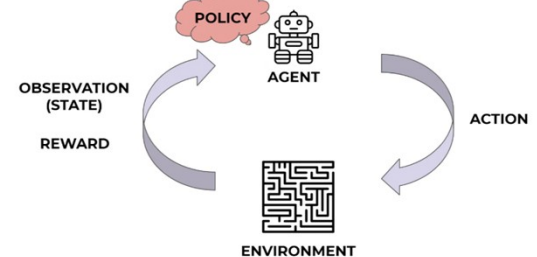
- Real environments may even have states that are difficult to define!



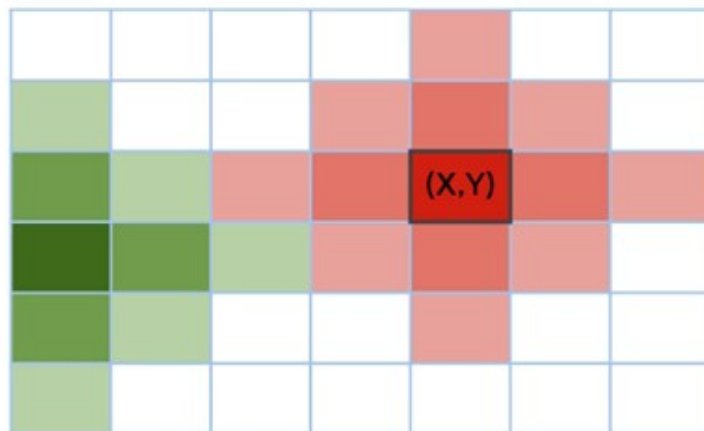
	Action 0	Action 1	...	Action N
State 0	Q	Q	...	Q
State 1	Q	Q	...	Q
...	...	...	...	...
State N	Q	Q	...	Q

Reinforcement Learning

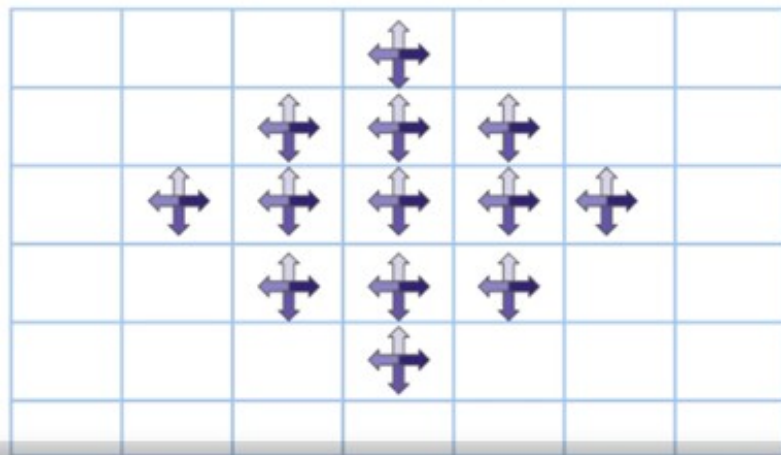
- Reinforcement Learning Cycle:



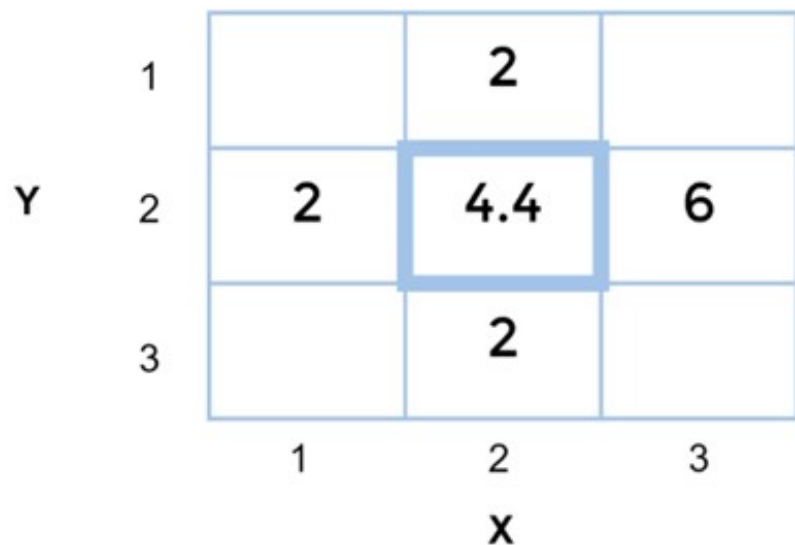
- Create a value neural network:



- Create a Policy Neural Network



- How to train the neural network?

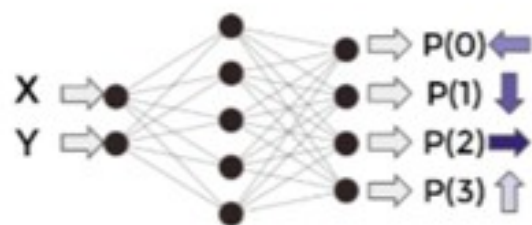
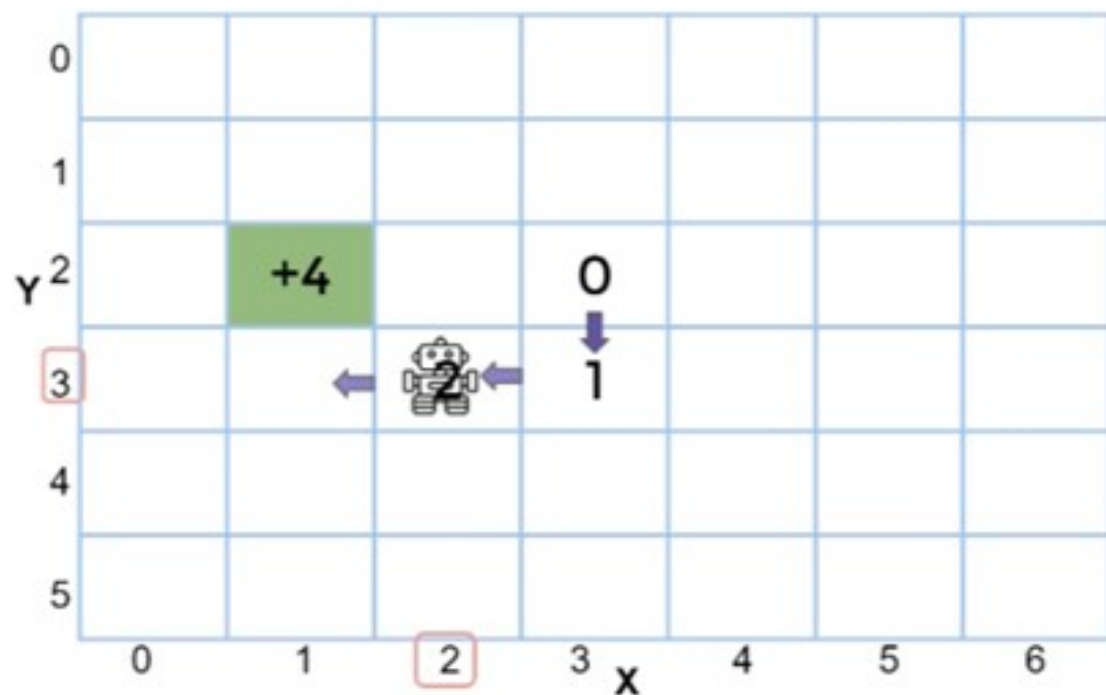


$$V(s) = \max_a (R(s, a) + \gamma V(s'))$$

Features		Label
X	Y	V

- Key Ideas:
 - Use Bellman Equation as basis of error metric for neural network.
 - Convert reinforcement learning process into a supervised learning problem for neural network to learn values from observations.

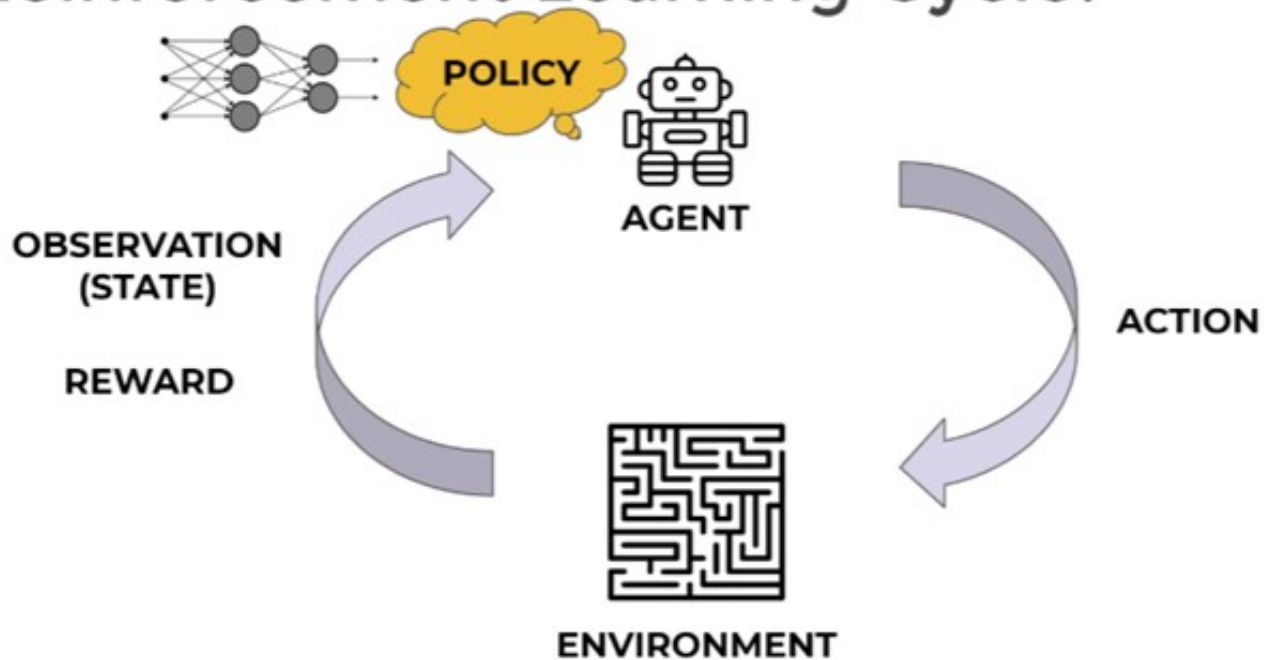
● Policy Network



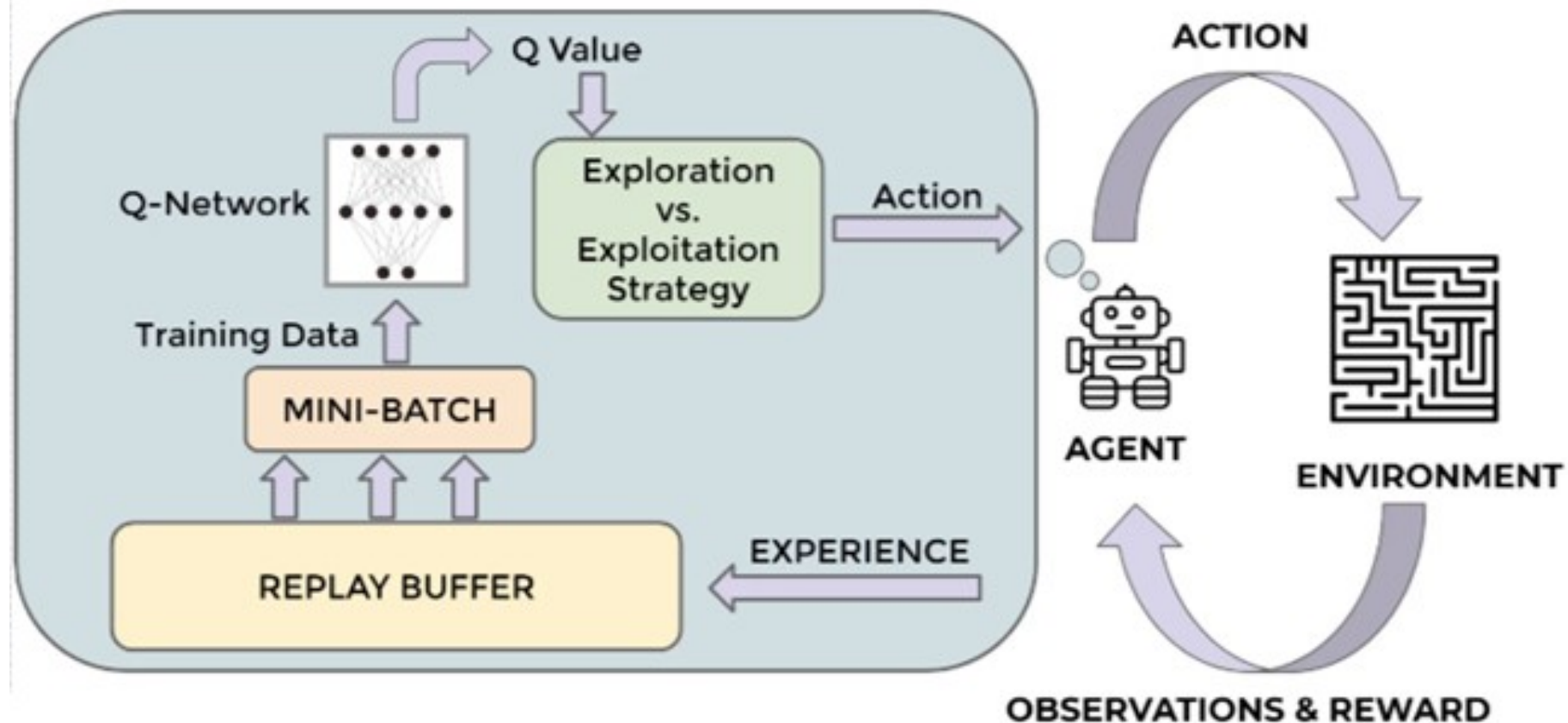
Gain	X	Y	Action
0	3	2	↓
1	3	3	←
2	2	3	←

Reinforcement Learning

- Reinforcement Learning Cycle:



Reinforcement Learning

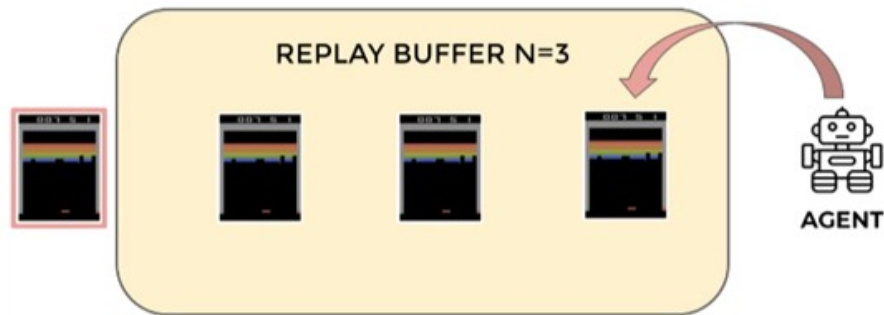


DQN Theory and Intuition

- I** • Data obtained from the basic reinforcement learning process is gathered “online”.

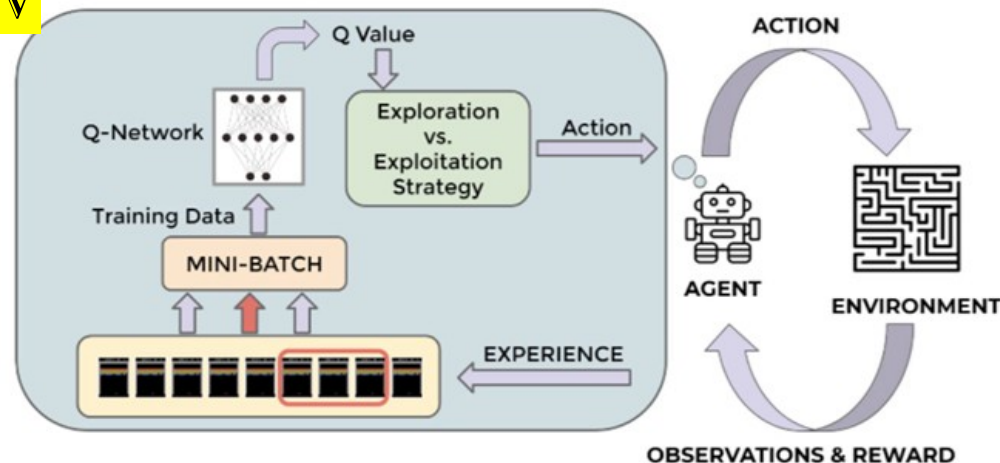


- III** • Experience Replay Buffer Deque:



- II** • Experience Replay Buffer:
- The buffer holds N steps of agent feedback from the environment (experiences).
 - The more complex the environment, the larger this buffer should be:
 - 10K-10M steps

IV



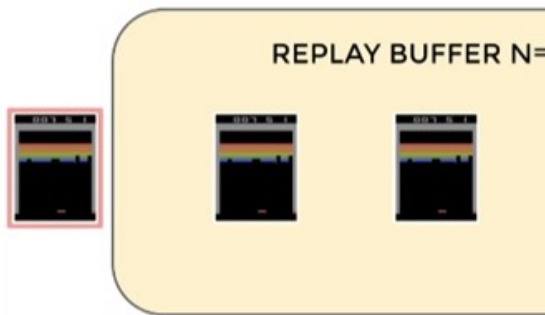
DQN Theory and Intuition

- I • Data from the reinforcement learning process is gathered “online”.



- II • Experience Replay Buffer: The buffer holds N steps of agent experience (experiences). The more complex the environment, the larger this buffer should be: 10M steps

- III • Experience Replay



```
for i in range(10):  
    deque_2.append(i)  
    print(deque_2)
```

```
deque([0, 1, 2, 3, 4], maxlen=5)
```

```
deque([0], maxlen=5)
```

```
deque([0, 1], maxlen=5)
```

```
deque([0, 1, 2], maxlen=5)
```

```
deque([0, 1, 2, 3], maxlen=5)
```

```
deque([0, 1, 2, 3, 4], maxlen=5)
```

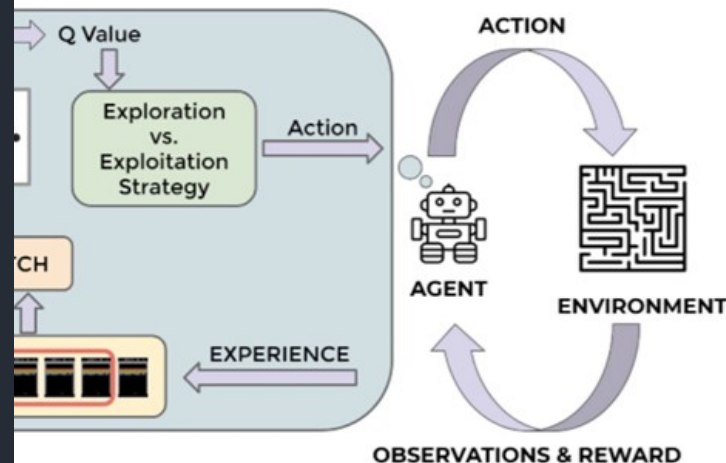
```
deque([1, 2, 3, 4, 5], maxlen=5)
```

```
deque([2, 3, 4, 5, 6], maxlen=5)
```

```
deque([3, 4, 5, 6, 7], maxlen=5)
```

```
deque([4, 5, 6, 7, 8], maxlen=5)
```

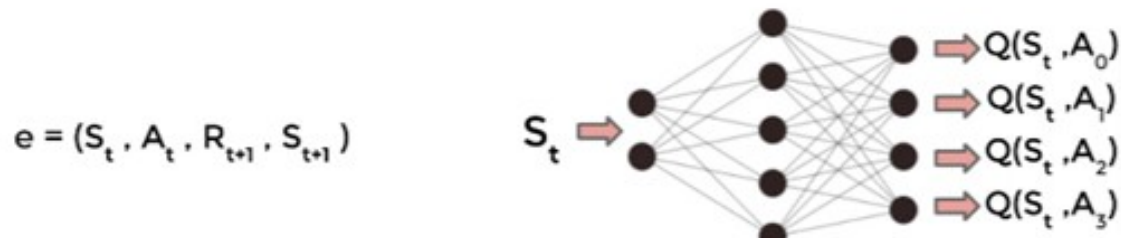
```
deque([5, 6, 7, 8, 9], maxlen=5)
```



DQN Theory and Intuition

- Finally, in practical terms, what information does the “experience” actually hold?
 - A tuple with the following:
 - $\text{exp} = (S_t, A_t, R_{t+1}, S_{t+1})$
-

- Q-Network Training Process:



DQN Theory and Intuition

- For example, every 100 steps copy Q-Network weights to Target Network.



DQN Theory and Intuition

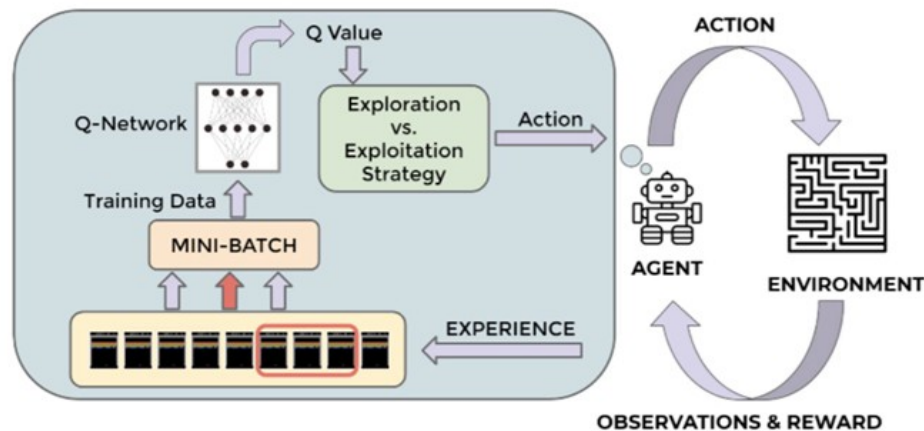
- Tasks to Complete:
 - Define Hyperparameters
 - Epsilon Greedy Action Selection
 - Understand Deque Object
 - Create Replay Function
 - Target Model Update Function

DQN Theory and Intuition

- There are many Deep RL libraries!
 - OpenAI Baselines
 - Stable Baselines
 - TF Agents
 - TensorForce
 - Keras-RL
 - Keras-RL2

DQN Theory and Intuition

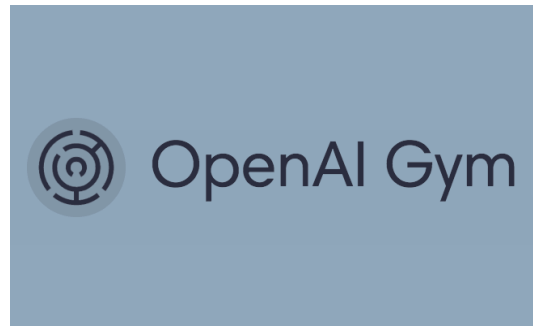
- Keras-RL2 (*keras-rl2*)
 - Separates out model, replay memory, policy, and agent.
 - Still a high level abstraction, but with a clear connection to the theoretical topics discussed in this course.



DQN: technical difficulties



Tensorflow 2.10.1



gym-0.26.2

Development > Data Science > Deep Learning

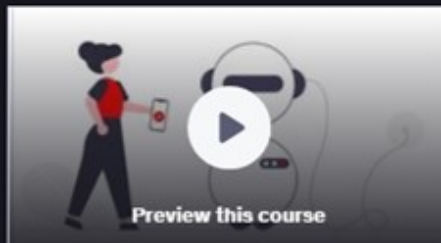
Practical AI with Python and Reinforcement Learning

Learn how to use Reinforcement Learning techniques to create practical Artificial Intelligence programs!

4.3 ★★★★★ (1,241 ratings) 12,780 students

Created by [Jose Portilla](#), [Pierian Training](#)

🕒 Last updated 04/2023 🌐 English 📺 English [Auto], Korean [Auto], [2 more](#)



i You purchased this course on
Mar. 05, 2025

[Go to course](#)

30-Day Money-Back Guarantee

This course includes:

- 📺 26.5 hours on-demand video
- 📄 6 articles
- 📎 9 downloadable resources
- 📱 Access on mobile and TV
- ∞ Full lifetime access
- 📜 Certificate of completion

[Share](#) [Gift this course](#) [Apply Coupon](#)

Training 5 or more people?

Get your team access to 30,000+ top Udemy courses anytime, anywhere.

[Try Udemy Business](#)

What you'll learn

- ✓ Reinforcement Learning with Python
- ✓ Using TensorFlow to create Convolution Neural Networks for Images
- ✓ Using OpenAI to create your own environments for any problem
- ✓ Tabular Q-Learning
- ✓ Deep Q-Learning (DQN)
- ✓ Cross Entropy Method for Reinforcement Learning
- ✓ Dueling DQN
- ✓ Creating Artificial Neural Networks with TensorFlow
- ✓ Using OpenAI to work with built-in game environments
- ✓ Create Artificially Intelligent Agents
- ✓ State-action-reward-state-action (SARSA)
- ✓ DQN using Convolutional Neural Networks
- ✓ Double DQN

[Show less](#) ^

What you'll learn

- ✓ Reinforcement Learning with Python
- ✓ Using TensorFlow to create Convolution Neural Networks for Images
- ✓ Using OpenAI to create your own environments for any problem
- ✓ Tabular Q-Learning
- ✓ Deep Q-Learning (DQN)
- ✓ Cross Entropy Method for Reinforcement Learning
- ✓ Dueling DQN
- ✓ Creating Artificial Neural Networks with TensorFlow
- ✓ Using OpenAI to work with built-in game environments
- ✓ Create Artificially Intelligent Agents
- ✓ State-action-reward-state-action (SARSA)
- ✓ DQN using Convolutional Neural Networks
- ✓ Double DQN

Show less ^

- ✓ Cross Entropy Method for Reinforcement Learning
- ✓ Dueling DQN

Show less ^

- ✓ Double DQN

Training 5 or more people?

Get your team access to 30,000+ top Udemy courses anytime, anywhere.

[Try Udemy Business](#)

A simple problem



UCI MACHINE LEARNING AND 1 COLLABORATOR · UPDATED 9 YEARS AGO



4675

<> Code

Download



Pima Indians Diabetes Database

Predict the onset of diabetes based on diagnostic measures



Data Card

Code (3583)

Discussion (54)

Suggestions (0)

Example

- Keras RL2
- Tensorflow 2.10.1

```
class PimaEnv(Env):
    def __init__(self, X, y):
        super(PimaEnv, self).__init__()
        self.X = X
        self.y = y
        self.max_steps = 10
        self.current_step = 0
        self.current_index = 0
        self.indices = []

        self.observation_space = Box(low=0, high=1, shape=(X.shape[1],), dtype=np.float32)
        self.action_space = Discrete(2)

    def reset(self):
        self.current_step = 0
        self.indices = np.random.permutation(len(self.X))[:self.max_steps]
        self.current_index = 0
        return self.X[self.indices[self.current_index]]

    def step(self, action):
        reward = 1 if action == self.y[self.indices[self.current_index]] else 0
        self.current_index += 1
        self.current_step += 1
        done = self.current_step >= self.max_steps

        if not done:
            obs = self.X[self.indices[self.current_index]]
        else:
            obs = np.zeros(self.X.shape[1]) # dummy

        return obs, reward, done, {}
```

Customized Environment

Example

- Keras RL2
- Tensorflow 2.10.1
- gym-0.26.2

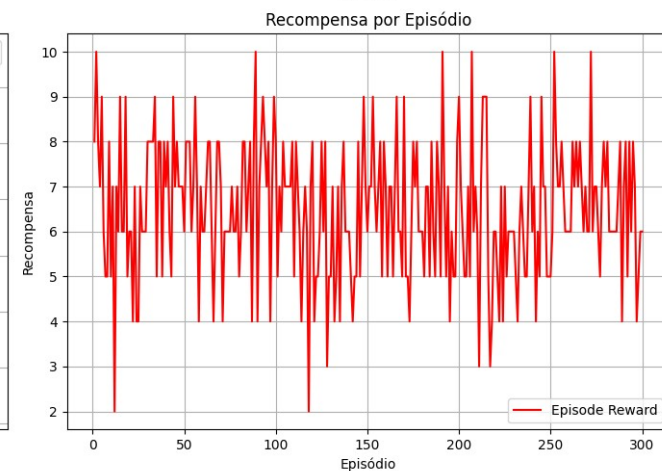
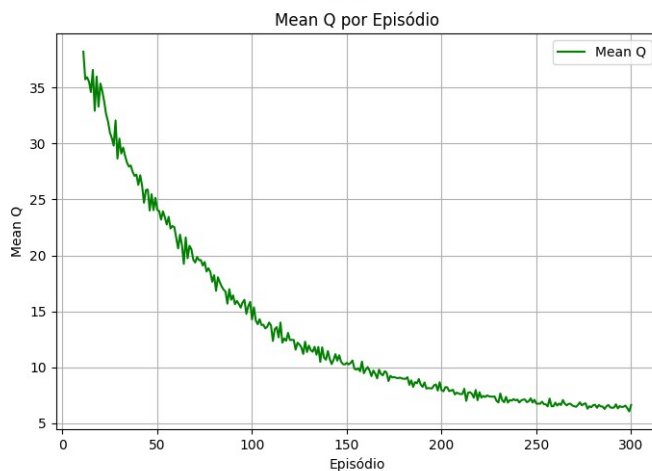
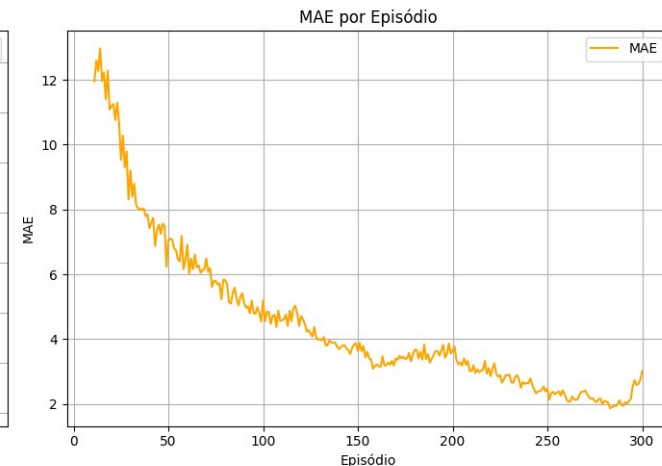
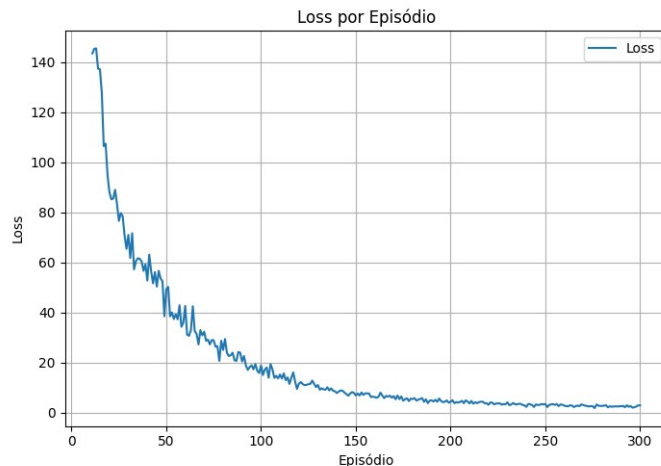
```
model = Sequential()  
model.add(InputLayer(input_shape=(1, X.shape[1])))  
model.add(Flatten())  
model.add(Dense(24, activation='relu'))  
model.add(Dense(24, activation='relu'))  
model.add(Dense(nb_actions, activation='linear'))
```

```
memory = SequentialMemory(limit=5000, window_length=1)  
policy = BoltzmannQPolicy()  
  
dqn = DQNAgent(model=model,  
               nb_actions=nb_actions,  
               memory=memory,  
               nb_steps_warmup=100,  
               target_model_update=1e-2,  
               policy=policy)  
dqn.compile(Adam(learning_rate=1e-3), metrics=['mae'])
```

```
dqn.fit(env,  
        nb_steps=3000,  
        visualize=False,  
        verbose=2,  
        callbacks=[logger])
```

Example

- Keras RL2
- Tensorflow 2.10.1
- gym-0.26.2



Conclusions

- Reinforcement learning (RL) has a natural application for interaction-based learning; therefore, it can, in principle, be used for online real-time fraud detection.
- On the other hand, from the literature, we know that RL does not present good scores in comparison with machine learning models (ML) such as boosted decision trees.
- Due to that, an investigation about the usage of Quantum machine learning (QML) can improve the RL for this task (at least in the anomaly detection area).

Conclusions

- Reinforcement learning (RL) has a natural application for interaction-based learning; therefore, it can, in principle, be used for online real-time fraud detection.
- On the other hand, from the literature, we know that RL does not present good scores in comparison with machine learning models (ML) such as boosted decision trees.
- Due to that, an investigation about the usage of Quantum machine learning (QML) can improve the RL for this task (at least in the anomaly detection area).
- A first step was achieved, a first RL model were developed and roughly tested.

Conclusions

- The next steps are:
 - Fraud data set tests;
 - Usage of other packages such as pytorch;
 - Replication of some interesting paper applying RL to fraud detection and comparing with the state of the art of fraud detection ML methodology (classical);
 - Start mixing the classical with Quantum elements of ML into Classical RL structure.